

Corporación Universitaria
Politécnico Costa Atlántica

**PROGRAMACIÓN WEB-BACKEND.**

**PROYECTO ARQUITECTURA HEXAGONAL: AUTORENT.**


**INTEGRANTES:**

**ISA MARIA MACHUCA DE LA ROSA.**

**DOCENTE:**

**IVÁN DEVIA.**

**POLITÉCNICO DE LA COSTA ATLÁNTICA**
**BARRANQUILLA/ATLÁNTICO**


**2026**

# PROYECTO ARQUITECTURA HEXAGONAL: AUTORENT.

## INTRODUCCIÓN

En este documento se presentan las evidencias del funcionamiento del proyecto AutoRent, una aplicación para gestionar el alquiler de autos, desarrollada con arquitectura hexagonal en Java con Spring Boot y H2.

Las evidencias se tomaron de tres formas: desde la consola de H2, para ver los datos que se guardan en las tablas; desde el navegador, consultando los endpoints con el método GET; y desde Postman, donde se creó una colección para validar las pruebas.

Primero se muestran los datos precargados en H2, luego las pruebas del 1 al 9 en el orden pedido por el docente, después un caso exitoso (PLUS) y un caso de error, y al final un resumen de los resultados.

## CONTENIDO

**Datos precargados en H2** . . . . . **3**

**Pruebas en Postman** . . . . . **6**

1. Crear un auto . . . . . 6
2. Consultar el auto . . . . . 7
3. Crear un cliente . . . . . 8
4. Crear un alquiler . . . . . 10
5. Comprobar que el auto pasó a: DISPONIBLE → ALQUILADO . . . . . 11
6. Intentar alquilar nuevamente el mismo auto. Debe producir un error . . . . . 12
7. Finalizar el alquiler . . . . . 13
8. Comprobar que el auto regresó a: ALQUILADO → DISPONIBLE . . . . . 13
9. Intentar finalizar nuevamente el alquiler. Debe producir un error . . . . . 14
10. Evidencias caso exitoso . . . . . 15
11. Evidencias caso error . . . . . 15

**Resumen de resultados** . . . . . **16**

**Conclusión** . . . . . **16**

## DATOS PRECARGADOS EN H2

**Nota: la base de datos se genera con unos datos iniciales precargados en memoria, que son los que muestra H2 al levantar el servidor (1 auto, 1 cliente y 1 alquiler).**

Cada dato precargado se comprueba de dos formas: con una consulta en la consola de H2 y con su endpoint desde el navegador.

- **Auto precargado en H2:**

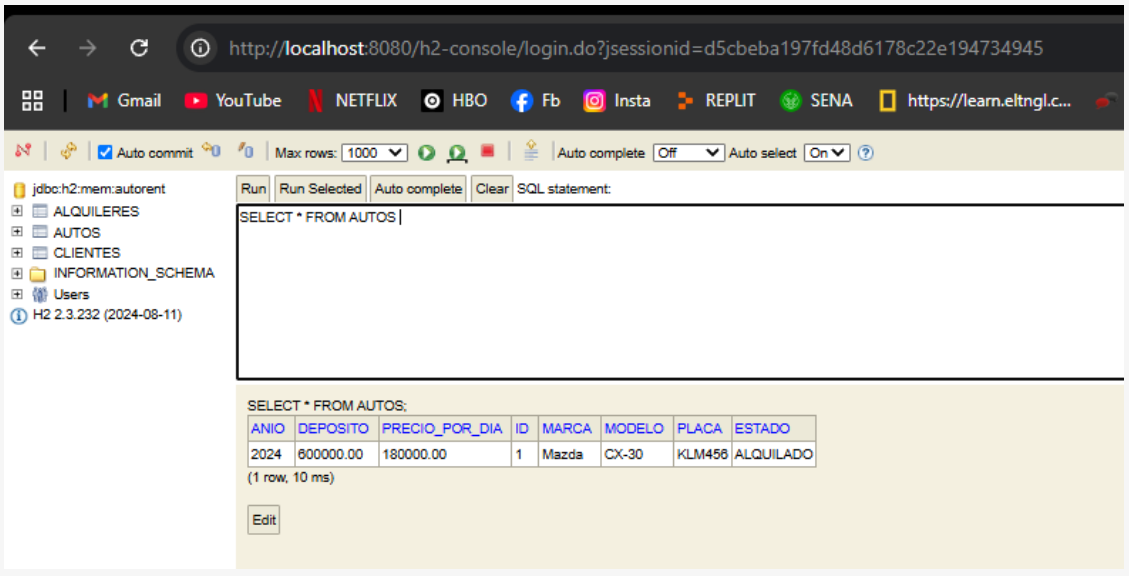



*Figura 1. Consulta SELECT * FROM AUTOS en H2: auto Mazda CX-30 (placa KLM456) precargado.*

- **Endpoint para consultarlo: desde "disponibles" mostrará un json vacío porque el auto está "ALQUILADO", entonces lo buscamos con su id {1}.**

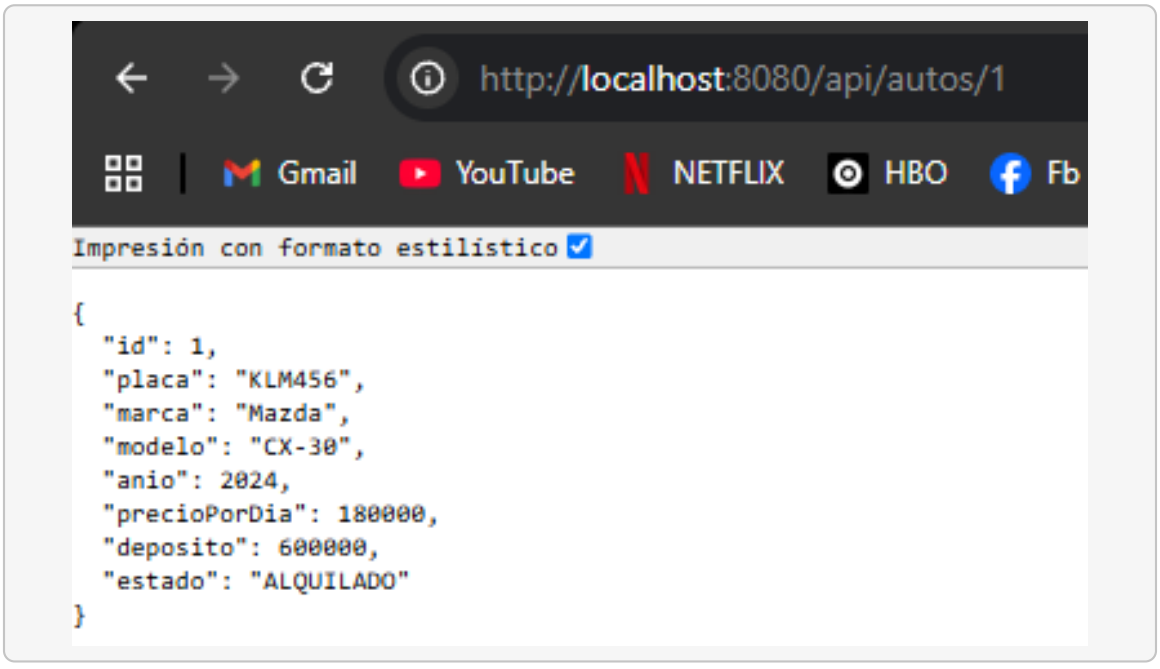



*Figura 2. GET /api/autos/1: el auto precargado está en estado ALQUILADO.*

- **Cliente precargado en H2:**

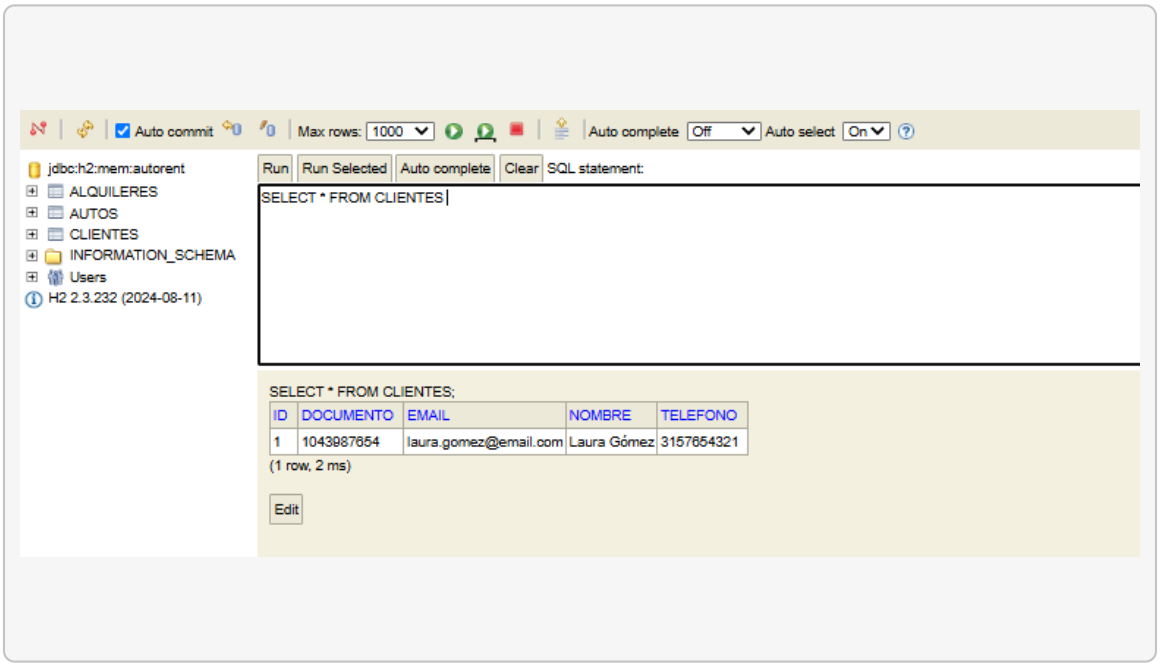



*Figura 3. Consulta SELECT * FROM CLIENTES en H2: cliente Laura Gómez precargado.*

- **Endpoint para consultarlo: se consulta el cliente con su id {1}:**

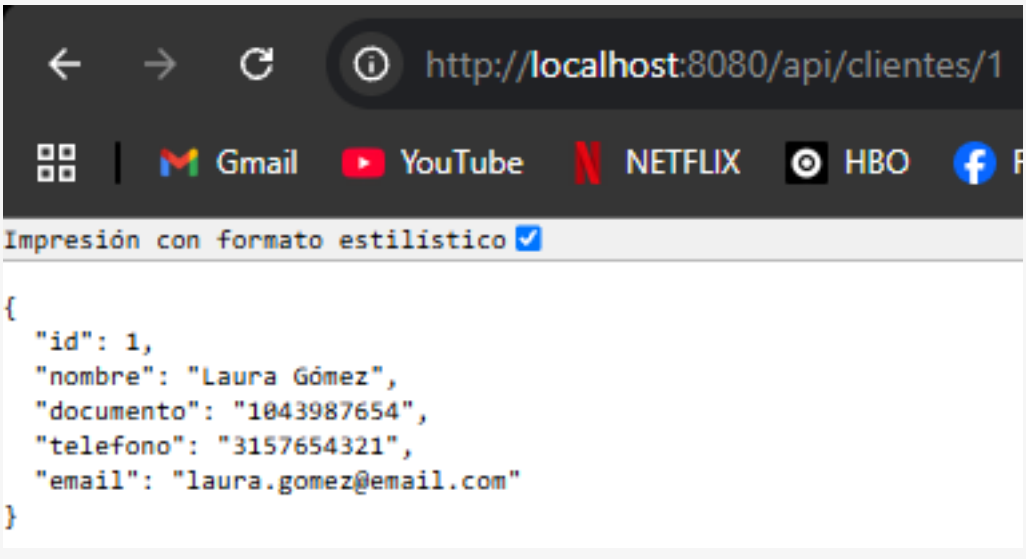



*Figura 4. GET /api/clientes/1: datos del cliente precargado.*

- **Alquiler precargado en H2:**

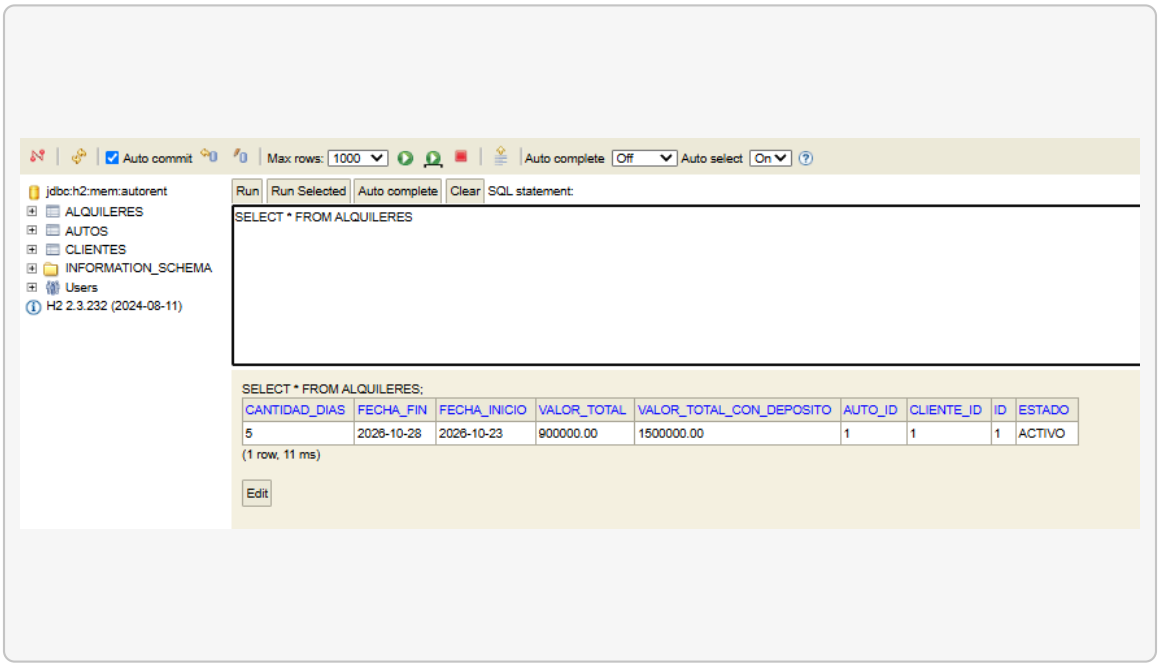



*Figura 5. Consulta SELECT * FROM ALQUILERES en H2: alquiler precargado, estado ACTIVO.*

- **Endpoint para consultarlo: se consulta el alquiler con su id {1}.**

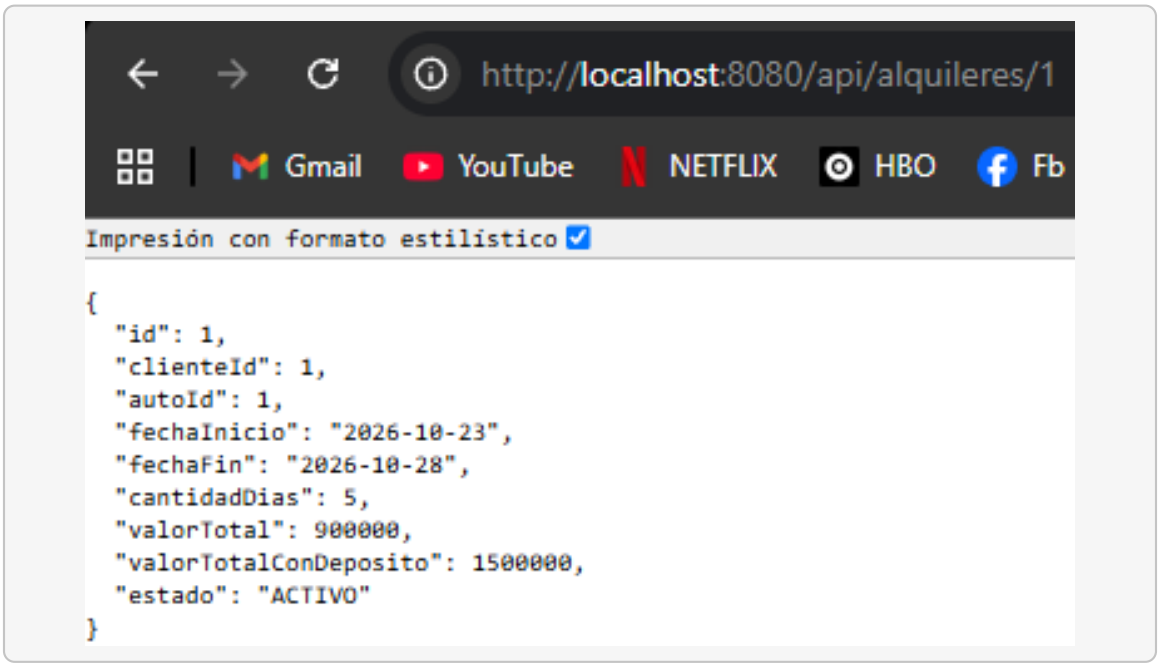



*Figura 6. GET /api/alquileres/1: alquiler precargado en estado ACTIVO.*

## PRUEBAS EN POSTMAN

**Nota: se creó una colección en Postman para validar las pruebas.**

La colección se llama “AutoRent - Hexagonal (v2, con datos precargados)”. En cada prueba se envía la petición desde Postman y, cuando la prueba cambia datos, el resultado también se comprueba en H2 y desde el navegador con su endpoint.

### 1. Crear un auto

Se registra un nuevo auto con sus datos, incluido el depósito. En la respuesta el auto queda con el estado DISPONIBLE.

- **Se crea el auto desde la petición: (POST)**

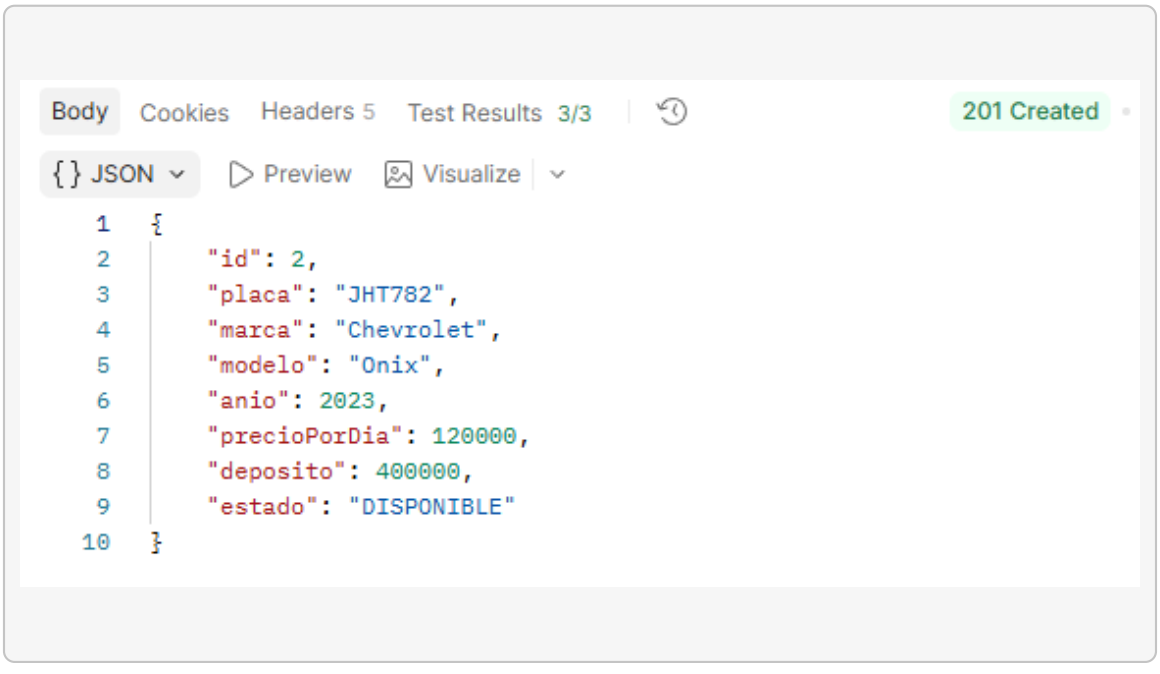



***Figura 7.*** *POST /api/autos: respuesta 201 Created, auto con id 2 y estado DISPONIBLE.*

- **Se consulta en H2:**

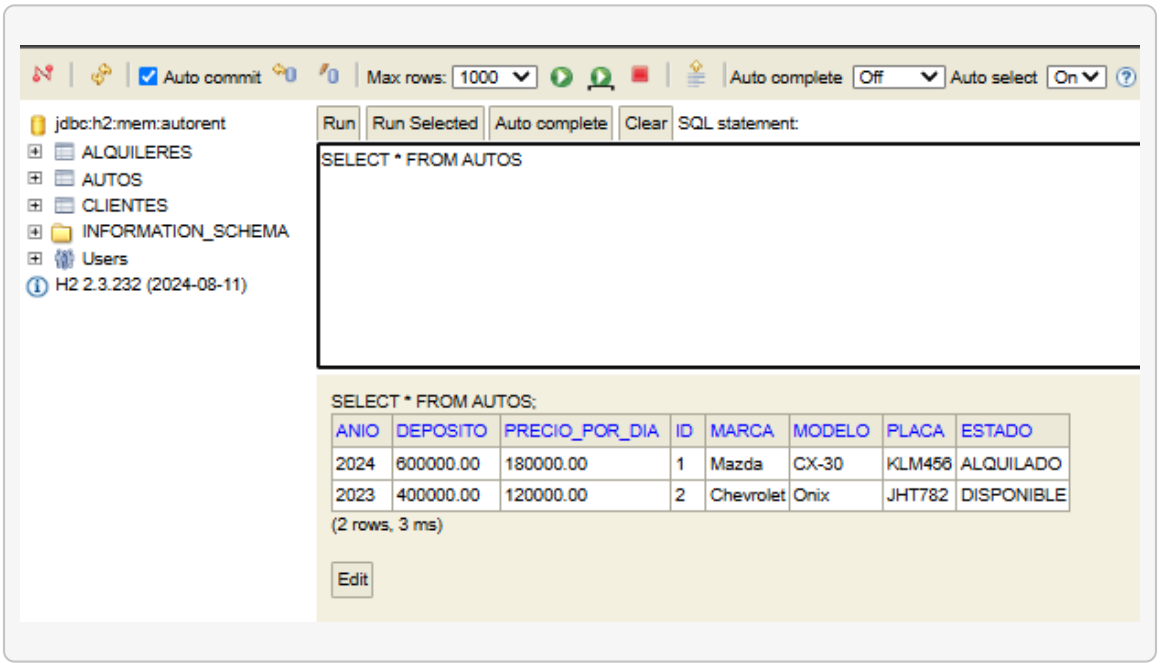



*Figura 8. Tabla AUTOS en H2: el auto creado (id 2) junto al auto precargado.*

## 2. Consultar el auto

Se verifica que el auto creado se pueda consultar por su id y que aparezca en la lista de autos disponibles.

- **Se consulta desde su endpoint: id{2}**

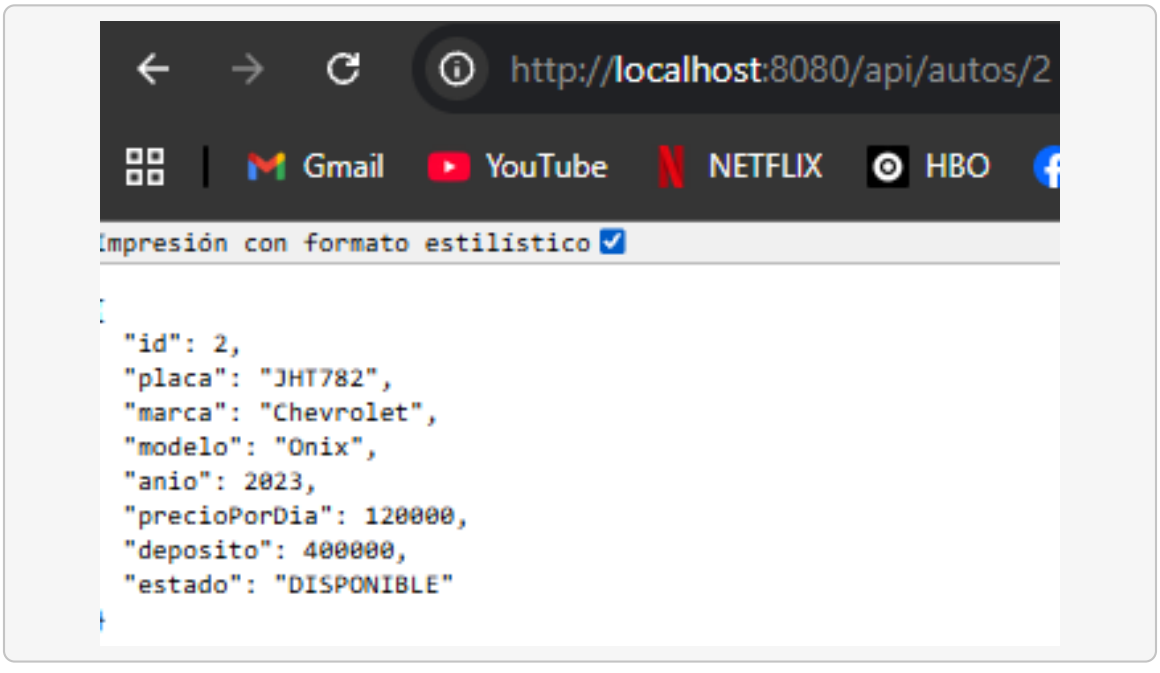



*Figura 9. GET /api/autos/2: el auto creado está DISPONIBLE.*

- **Se consulta desde "disponibles" porque el auto no está "ALQUILADO":**

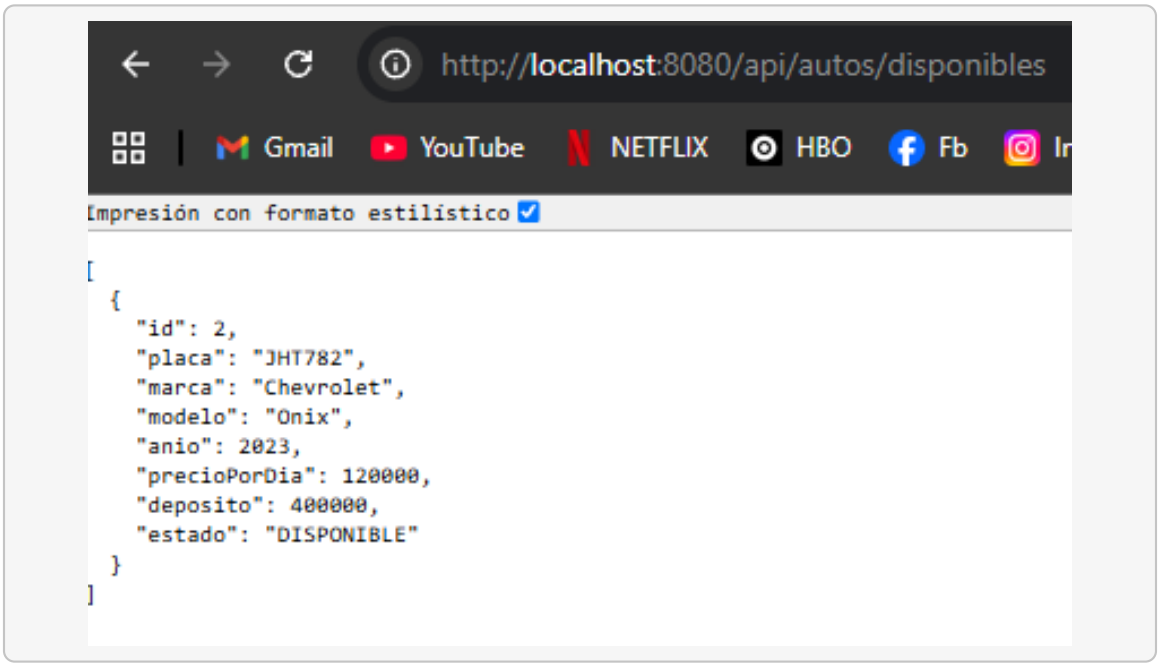



***Figura 10.*** *GET /api/autos/disponibles: solo aparece el auto 2.*

### 3. Crear un cliente

Se registra un nuevo cliente y se comprueba que quede guardado en la tabla CLIENTES.

- **Se crea el cliente desde la petición: (POST)**

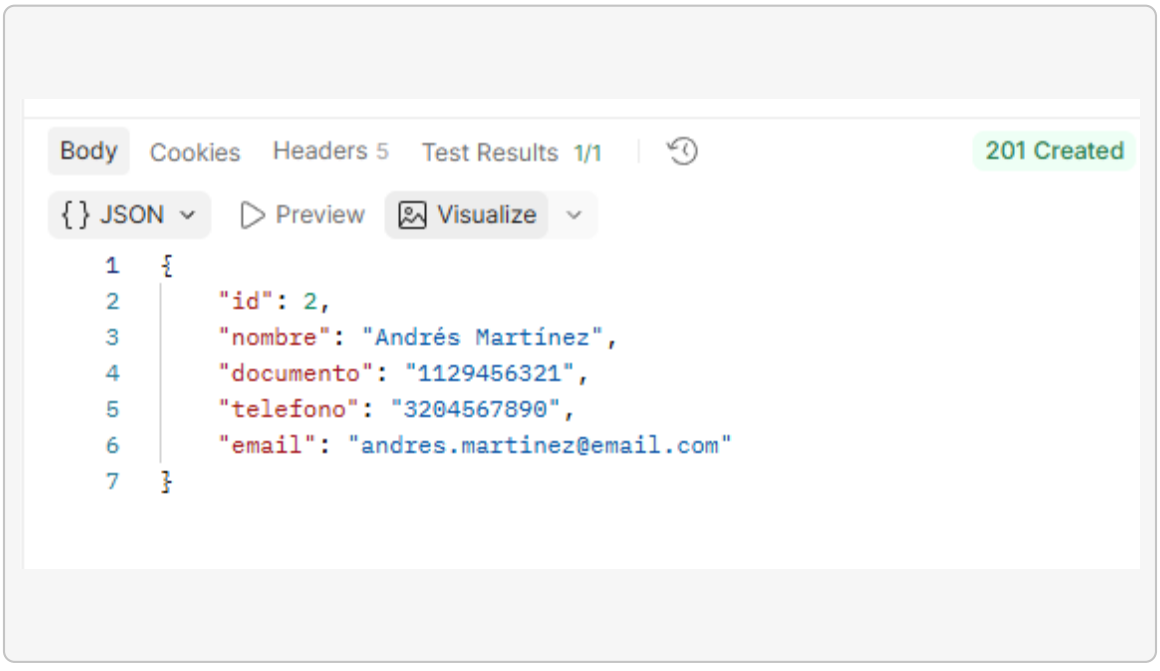



***Figura 11.*** *POST /api/clientes: respuesta 201 Created, cliente con id 2.*

- **Se consulta en H2:**

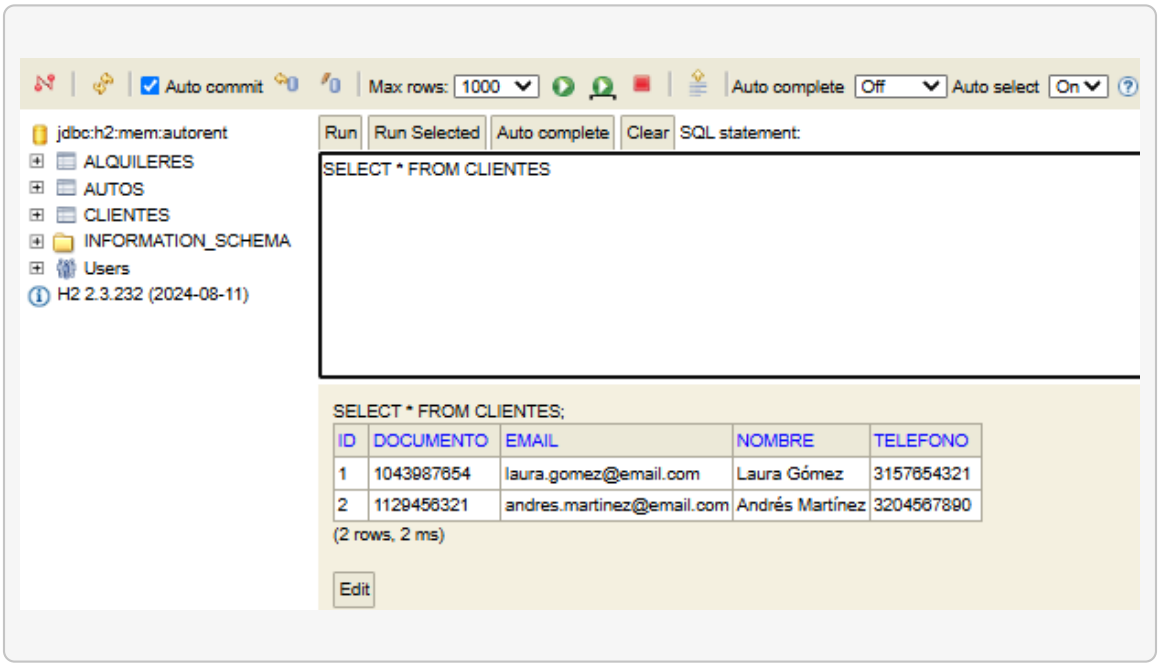



*Figura 12. Tabla CLIENTES en H2 con los dos clientes.*

- **Se consulta desde su endpoint: id{2}.**

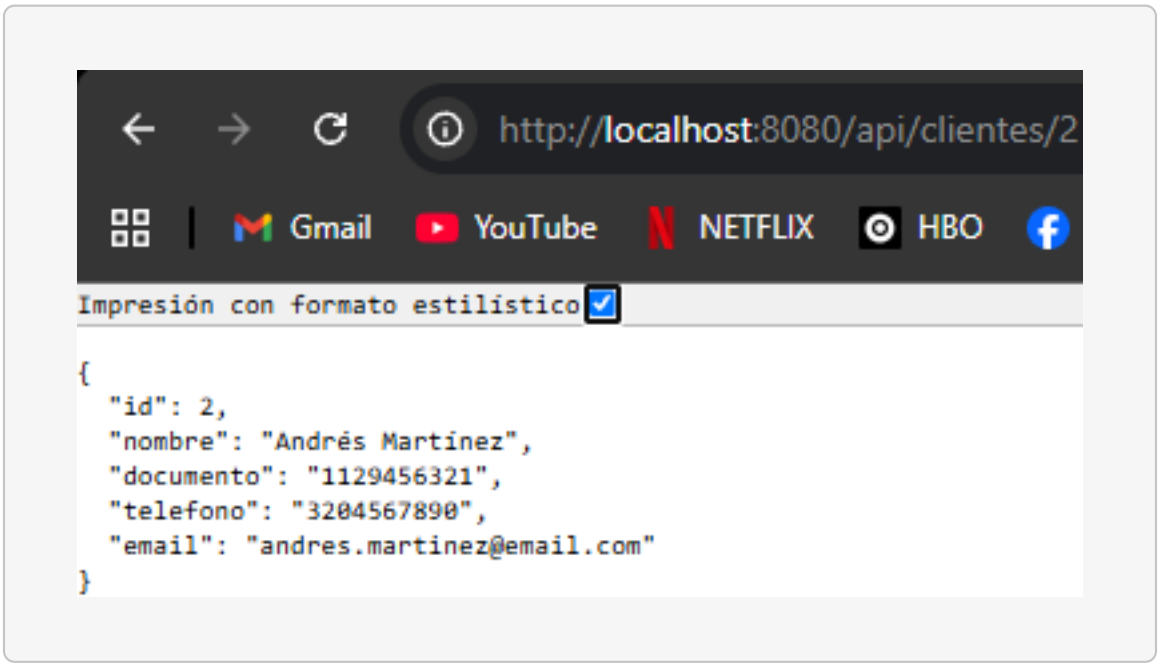



*Figura 13. GET /api/clientes/2: datos del cliente creado.*

### 4. Crear un alquiler

Se crea un alquiler para el cliente 2 y el auto 2 por 3 días. El valor total lo calcula el sistema (120000 x 3 = 360000) y el valor total con depósito suma el depósito del auto (360000 + 400000 = 760000).

- **Se crea el alquiler desde la petición: (POST)**

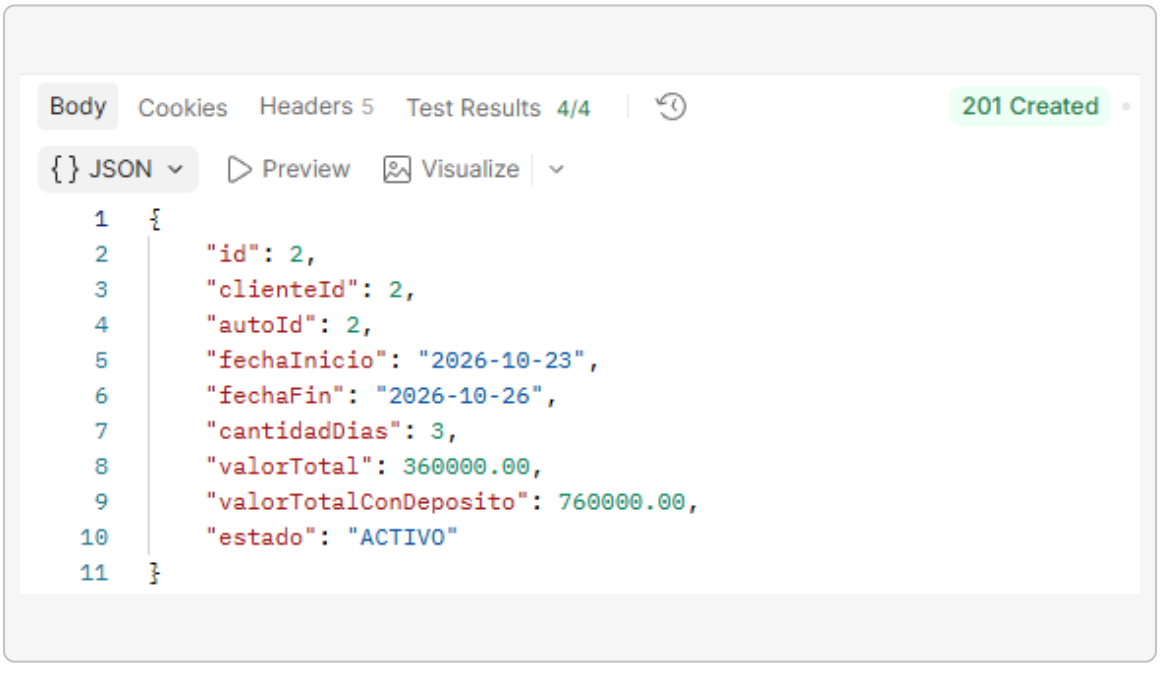



***Figura 14.*** *POST /api/alquileres: respuesta 201 Created, alquiler con id 2 en estado ACTIVO.*

- **Se consulta en H2:**

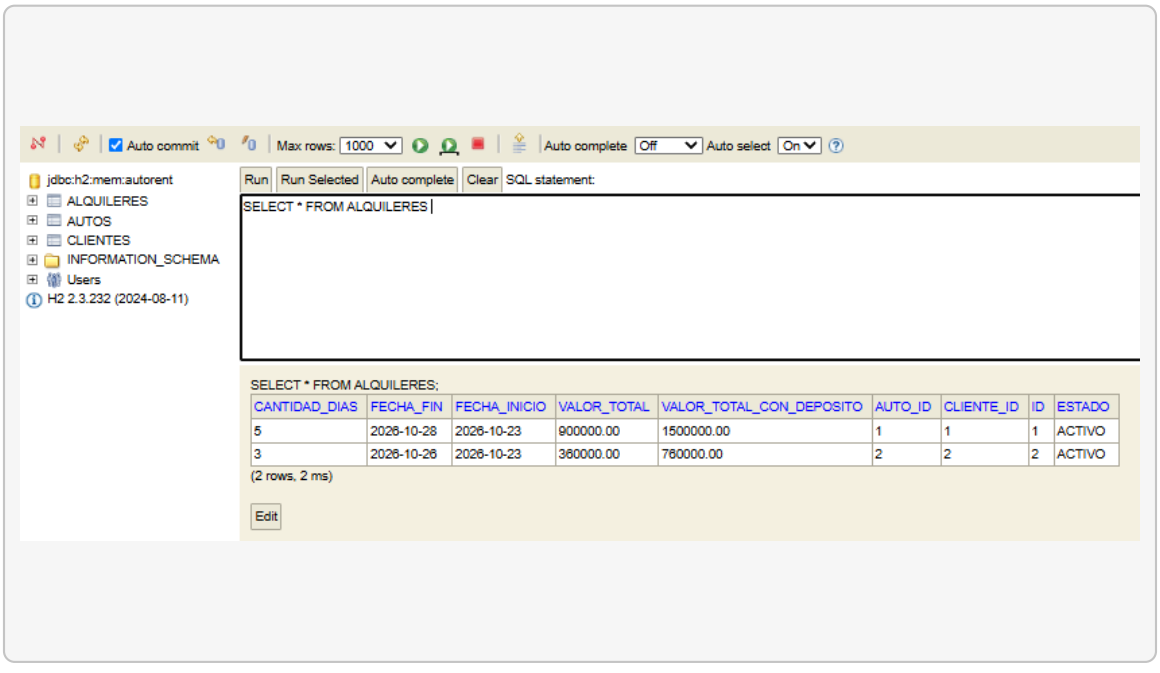



***Figura 15.*** *Tabla ALQUILERES en H2 con los dos alquileres.*

- **Se consulta desde su endpoint: id{2}.**

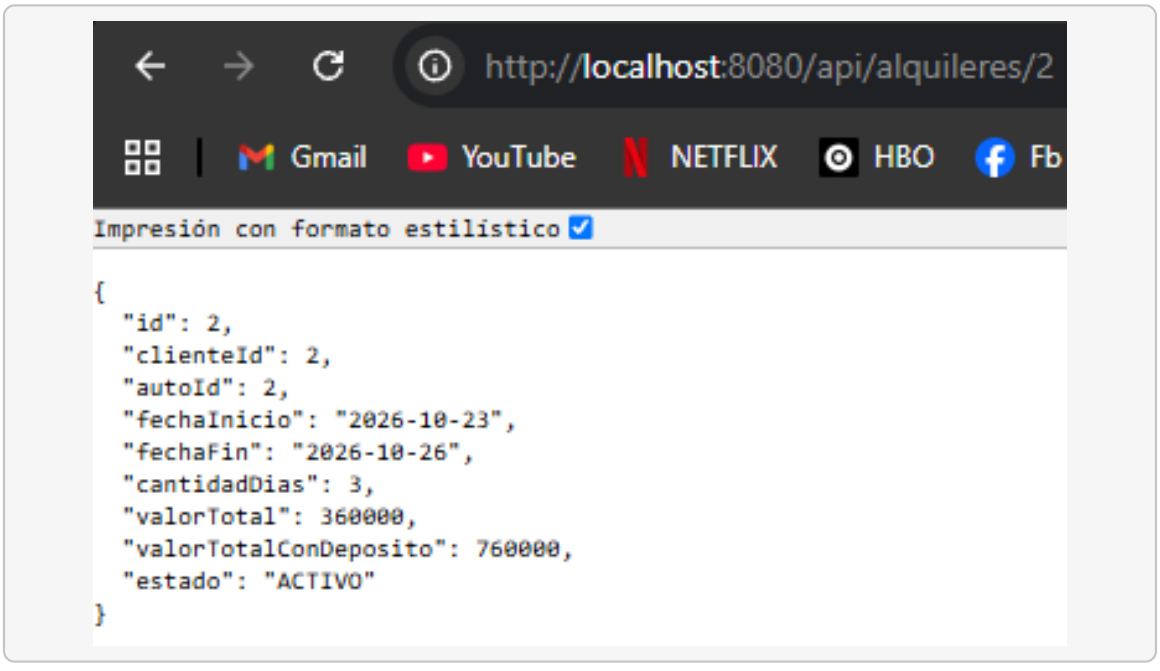



***Figura 16.*** *GET /api/alquileres/2: alquiler creado con valorTotal 360000.*

**5. Comprobar que el auto pasó a: DISPONIBLE → ALQUILADO**

Al crear el alquiler, el auto 2 debe cambiar de estado: de DISPONIBLE a ALQUILADO.

- **Se consulta el estado en H2:**

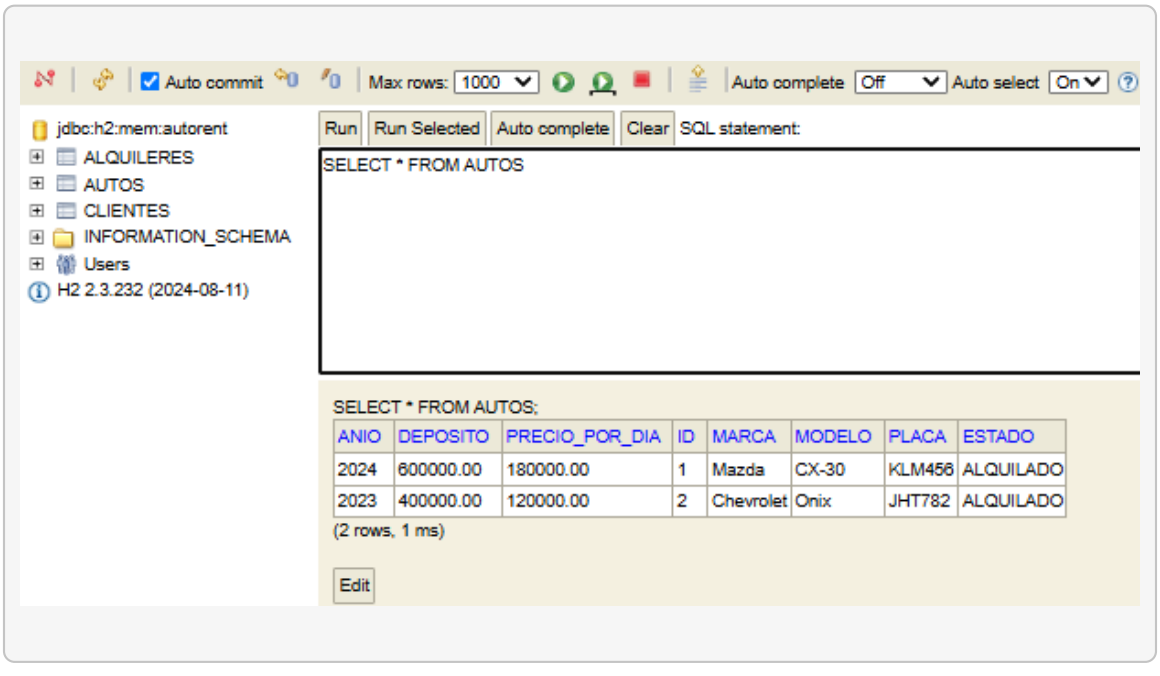



***Figura 17.*** *Tabla AUTOS en H2: el auto 2 pasó a ALQUILADO.*

- **Se consulta desde el endpoint: id{2}.**

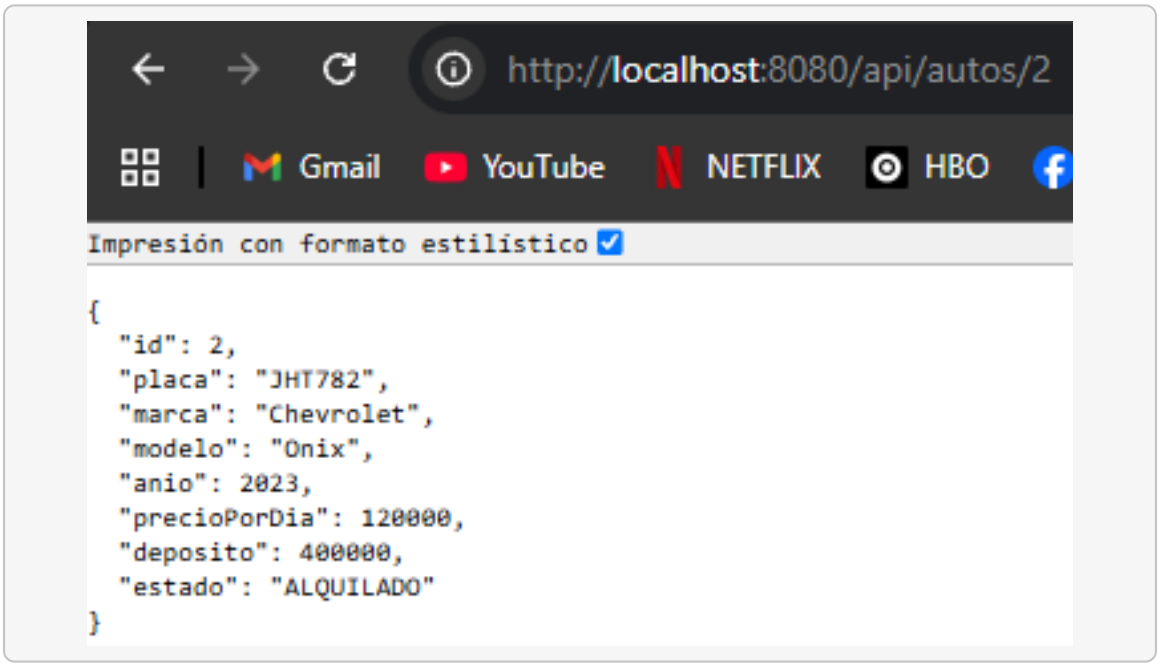



***Figura 18.*** *GET /api/autos/2: el auto está en estado ALQUILADO.*

### 6. Intentar alquilar nuevamente el mismo auto. Debe producir un error

Esta prueba valida la regla de negocio que no permite alquilar un auto que ya está ALQUILADO. Se espera un error.

- **Se envía la petición: (POST)**

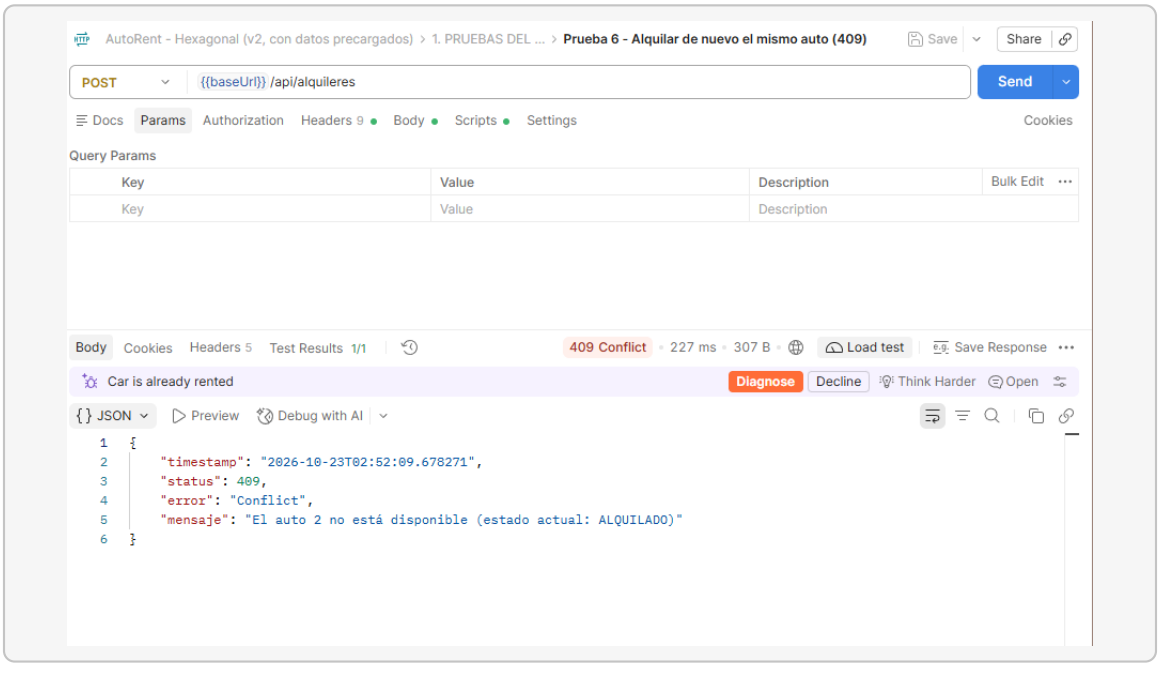



***Figura 19.*** *POST /api/alquileres: respuesta 409 Conflict, el auto no está disponible.*

### 7. Finalizar el alquiler

Se finaliza el alquiler 2 y el sistema cambia su estado a FINALIZADO.

- **Se envía la petición: (PUT)**

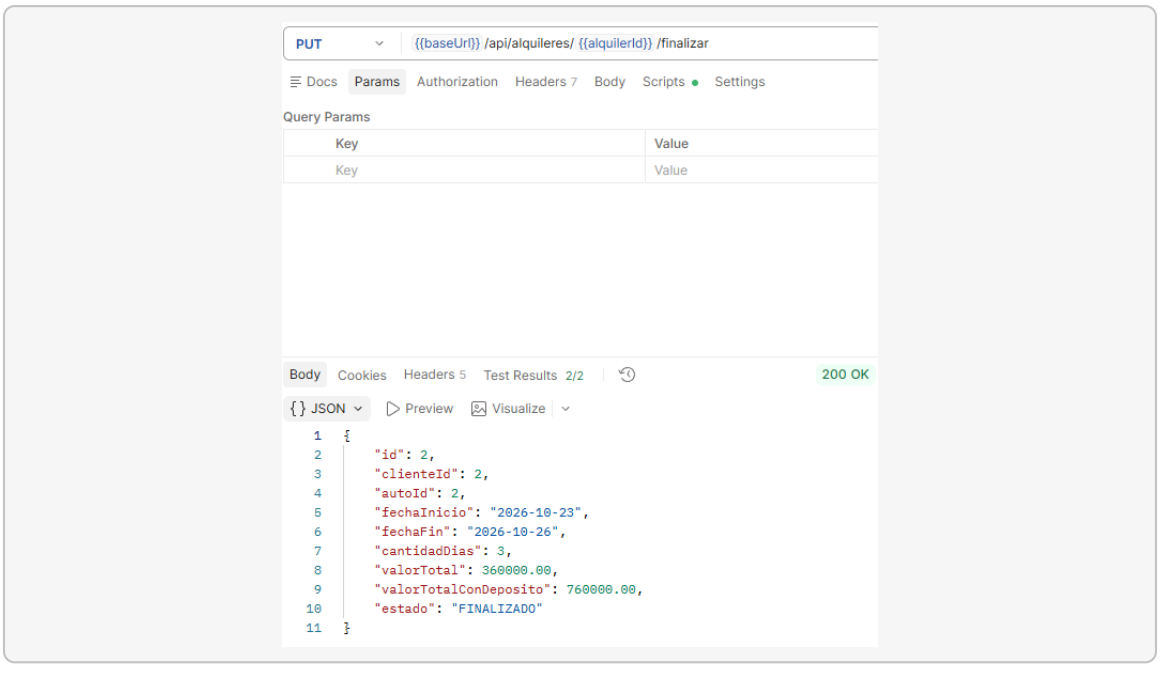



***Figura 20.*** *PUT /api/alquileres/{id}/finalizar: respuesta 200 OK, alquiler FINALIZADO.*

### 8. Comprobar que el auto regresó a: ALQUILADO → DISPONIBLE

Al finalizar el alquiler, el auto 2 debe volver a estar DISPONIBLE.

- **Se consulta desde H2:**

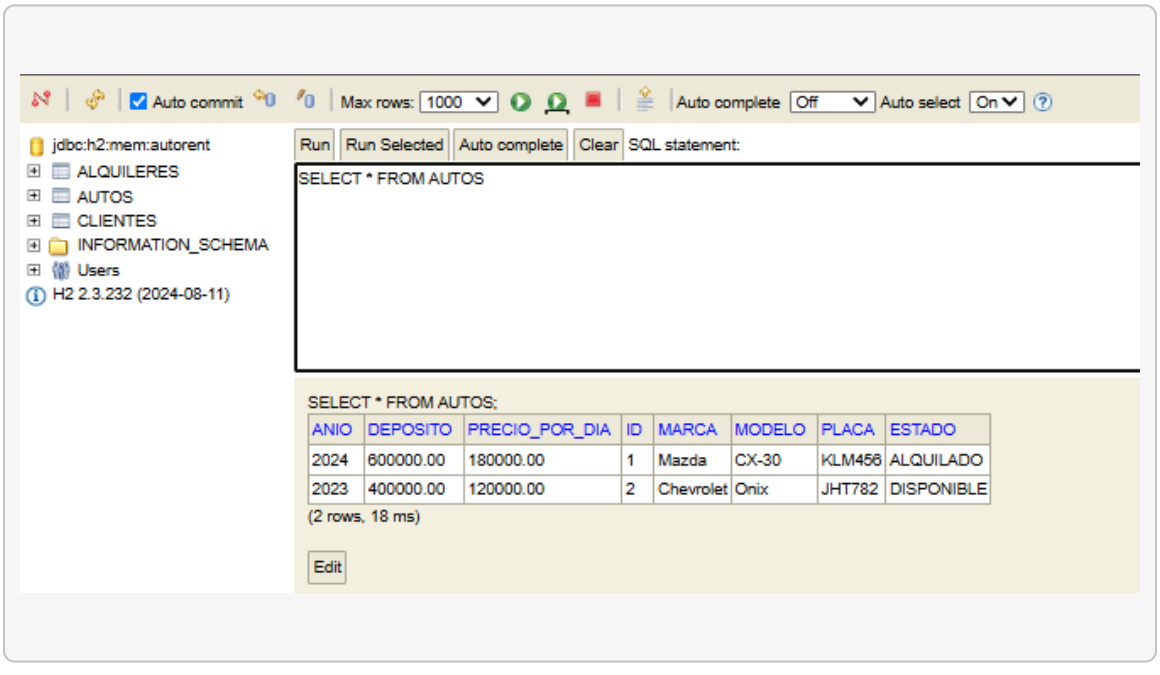



***Figura 21.*** *Tabla AUTOS en H2: el auto 2 volvió a DISPONIBLE.*

- **Se consulta desde su endpoint: id{2}**

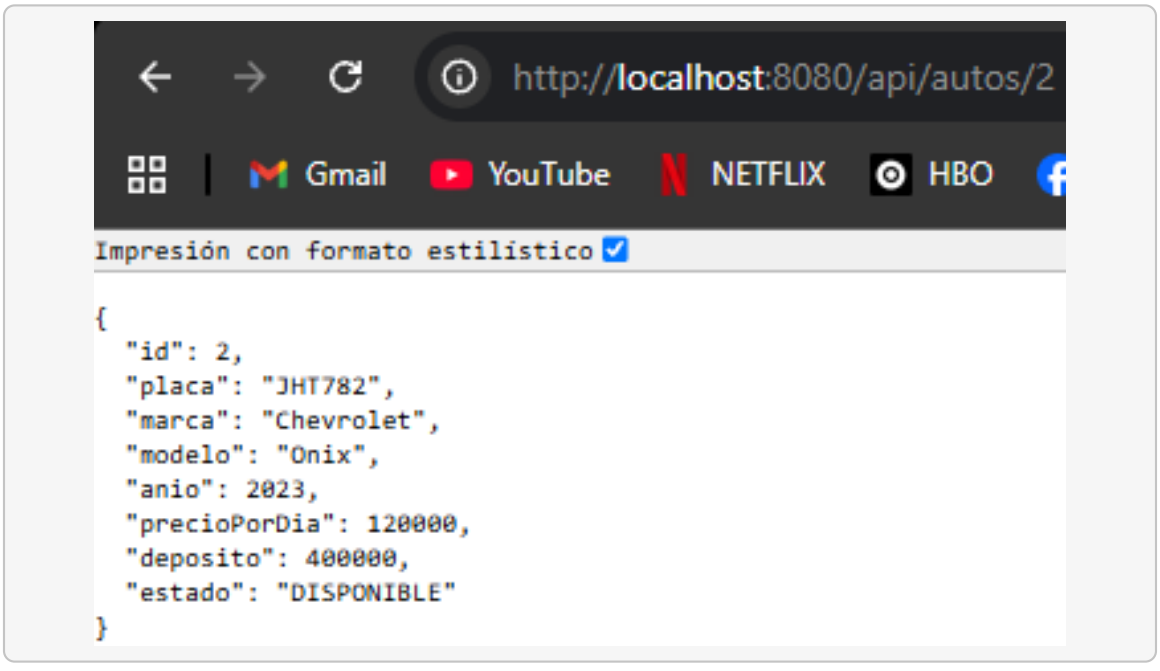



***Figura 22.*** *GET /api/autos/2: el auto está en estado DISPONIBLE.*

### 9. Intentar finalizar nuevamente el alquiler. Debe producir un error

Esta prueba valida la regla de negocio que no permite finalizar dos veces el mismo alquiler. Se espera un error.

- **Se envía la petición: (PUT)**

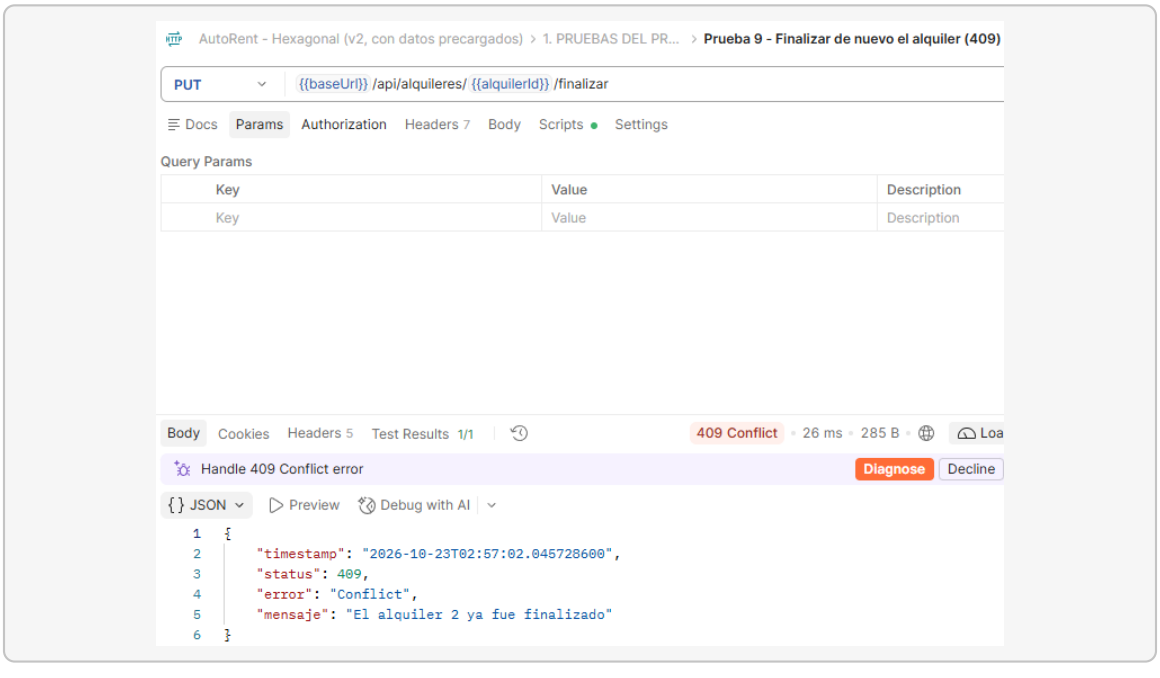



***Figura 23.*** *PUT /api/alquileres/{id}/finalizar: respuesta 409 Conflict, el alquiler ya estaba finalizado.*

## 10. Evidencias caso exitoso

Este caso exitoso corresponde a una consulta (PLUS) que agregué para que se pueda consultar el historial de alquileres por cliente teniendo en cuenta su id{2}.

- **Se hace la petición: (GET)**

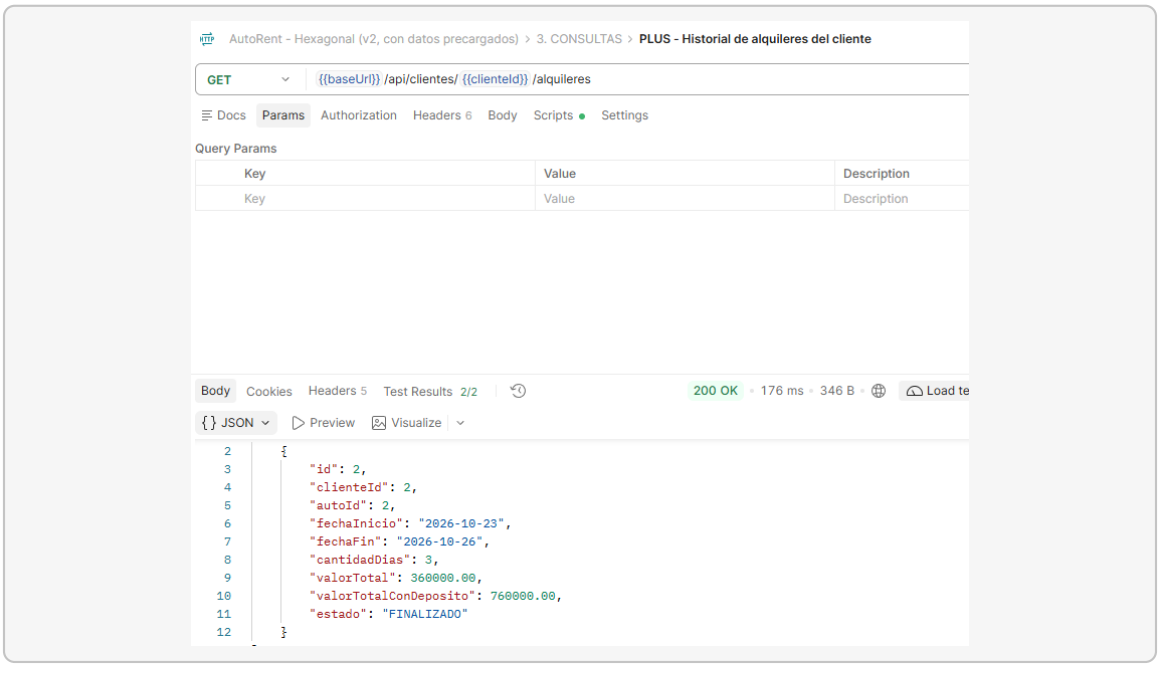



*Figura 24. GET /api/clientes/{id}/alquileres: respuesta 200 OK con el historial del cliente.*

## 11. Evidencias caso error

Este caso de error corresponde a una validación donde el alquiler tiene una cantidad de 0 días. Se espera que el sistema rechace la petición.

- **Se envía la petición: (POST)**

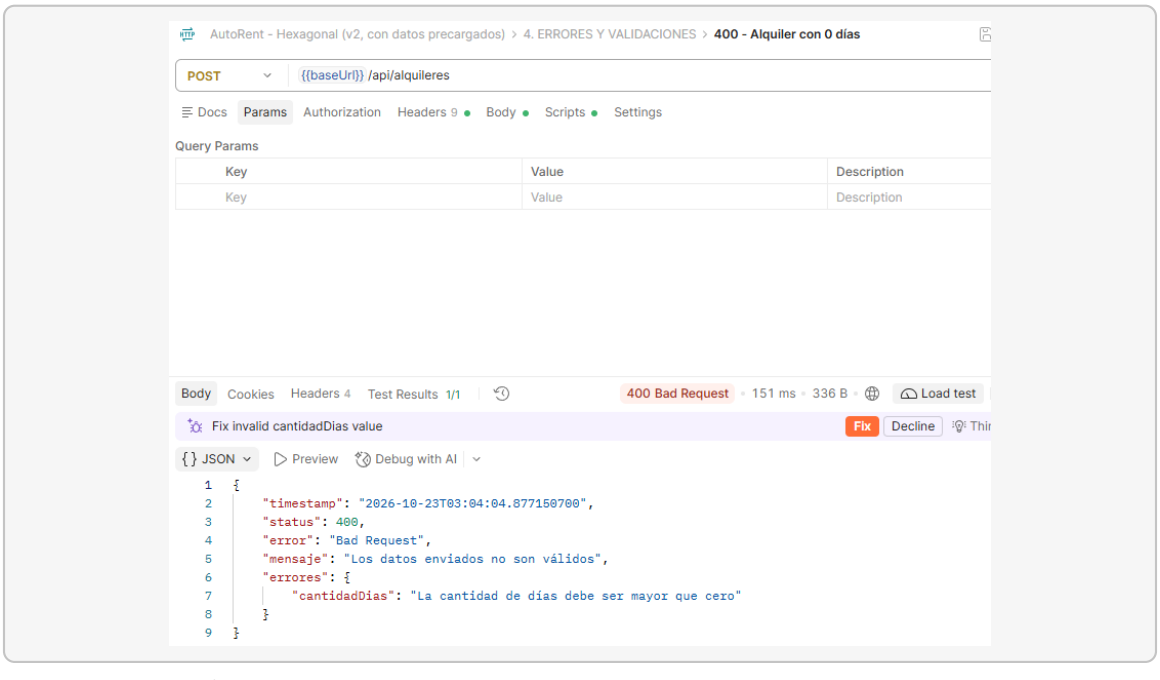



*Figura 25. POST /api/alquileres con 0 días: respuesta 400 Bad Request.*

## RESUMEN DE RESULTADOS

| N.º | Prueba | Petición | Resultado | Figuras |
|---|---|---|---|---|
| 1 | Crear un auto | POST /api/autos | 201 Created. Auto con id 2, estado DISPONIBLE | 7–8 |
| 2 | Consultar el auto | GET /api/autos/2<br>GET /api/autos/disponibles | Auto consultado; aparece en disponibles | 9–10 |
| 3 | Crear un cliente | POST /api/clientes | 201 Created. Cliente con id 2 | 11–13 |
| 4 | Crear un alquiler | POST /api/alquileres | 201 Created. Alquiler ACTIVO, valor total 360000 | 14–16 |
| 5 | Auto: DISPONIBLE → ALQUILADO | GET /api/autos/2 | Estado ALQUILADO | 17–18 |
| 6 | Alquilar de nuevo el mismo auto | POST /api/alquileres | 409 Conflict. Auto no disponible | 19 |
| 7 | Finalizar el alquiler | PUT /api/alquileres/2/finalizar | 200 OK. Alquiler FINALIZADO | 20 |
| 8 | Auto: ALQUILADO → DISPONIBLE | GET /api/autos/2 | Estado DISPONIBLE | 21–22 |
| 9 | Finalizar de nuevo el alquiler | PUT /api/alquileres/2/finalizar | 409 Conflict. Alquiler ya finalizado | 23 |
| 10 | Caso exitoso (PLUS): historial por cliente | GET /api/clientes/2/alquileres | 200 OK. Lista de alquileres del cliente | 24 |
| 11 | Caso de error: alquiler con 0 días | POST /api/alquileres | 400 Bad Request. Cantidad de días inválida | 25 |

## CONCLUSIÓN

Con las evidencias se comprueba que el proyecto AutoRent funciona como se esperaba: los datos se guardan en H2, el estado del auto cambia a ALQUILADO al crear un alquiler y vuelve a DISPONIBLE al finalizarlo, y el sistema responde con error 409 cuando el auto no está disponible o el alquiler ya fue finalizado, y con error 400 cuando la cantidad de días no es válida.

Además, la consulta adicional (PLUS) permite ver el historial de alquileres de un cliente.